

参考答案

一、(40 分) 评分标准：每小题 4 分，如有 N 个输出，每个占 4/N 分

题号	答案	备注
1	k=10 k=8 k=6, n=6	
2	lv po rm i	
3	54345	
4	a=5 a*b=0 a*b=6 a*b=4	
5	2,0,1,6	
6	20,30	
7	0,1	
8	49,0	
9	ABCdefh ABCd ABC AB	
10	201,201601,0,50000	

二、改错题 (20 分)

共 11 个错，找到 1 个算 1 分，改对算 1 分，总分 20 分，即允许 1 个没找到。

需找到错误，并明确指明错误所在，或对其进行修改，才算找到错误。

仅仅写第*行有误，不算找到错误。

1、第 1 (2 或 3) 行，增加 `#include <stdlib.h>` 或 `#include "stdlib.h"`

2、第 9 行，增加 `;`

3、第 14 行，`fp` 之前与 `==` 之前增加 `()`

4、第 21 行后 (或 22 行前) 增加 `fclose(fp);`

5、第 26 行，增加 `struct`

6、第 34 行 `%s` 改为 `%lf` 或者 `%f`

7、第 40 行 `i=1` 改为 `i=0`

8、第 43 行 改为 `strcmp(employ[i].position,"manager")==0`

9、第 45 行 `employ[i]->salary` 改为 `employ[i].salary`

10、第 48 行 `return` 改为 `return 0;`

11、第 23 行或其他相应正确的地方，增加函数声明 `void add(struct Emp employ[]);` 或将该函数调至 `main` 函数之前。（因为此错误所在的范围不好确认，此错误修改即算 2 分）

三、编程题

1. (12 分)

```
#include <stdio.h>
#include <string.h>                                     //指定头文件 1 分
int isRotateStr(char * s1, char * s2)
{
    char locals1[256];
    int iLen1, iLen2, i, j;
    char cLast;

    if (s1 == NULL || s2 == NULL)                       //参数错误检查 1 分
    {
        //提示参数出错
        return 0;
    }

    iLen1 = strlen(s1); iLen2 = strlen(s2);

    if (iLen1 != iLen2) return 0;                         //两个串不等长直接判断为假 1 分

    if (iLen1 == 0) return 1;                             //两个串都为空串直接判断为真 1 分

    if (strcmp(s1, s2) == 0) return 1;                    //两个串相等直接判断为真 1 分

    strcpy(locals1, s1);                                  //使用内部数组防止破坏字符串 1 分

    i = 1;

    while (i < iLen1)                                     //循环次数控制正确得 2 分
    {
        cLast = locals1[iLen1 - 1];
        for (j = iLen1 - 1; j > 0; j--)                   //字符串循环右移代码正确得 3 分; 若
            locals1[j] = locals1[j - 1];                 //写成循环左移, 代码正确也得 3 分

        locals1[0] = cLast;

        i++;

        if (strcmp(locals1, s2) == 0) return 1;          //1 分
    }

    return 0;
}
```

2. (1) (2 分)

```
struct Time {  
    int H, M;  
};
```

(2) (3 分)

```
void Cal( struct Time *Pt, struct Time *Gt, struct Time *Mt) {  
    int t=0;  
    Mt->M = Pt->M + Gt->M;  
    if ( Mt->M >= 60) {  
        Mt->M %= 60;    //或 Mt->M -=60;  
        t = 1;  
    }  
    Mt->H = (Pt->H + Gt->H + t) % 24;  
}
```

(3) (5 分)

```
void Sort( struct Time Cs[ ], int n) {  
    int i, j;  
    struct Time temp;  
    for (i=0; i<n-1; i++) {  
        for (j=0; j< n-1-i; j++)  
            if (Cs[j].H>Cs[j+1].H  
                || Cs[j].H==Cs[j+1].H && Cs[j].M>Cs[j+1].M) {  
                temp = Cs[j];  
                Cs[j]=Cs[j+1];  
                Cs[j+1]=temp;  
            }  
    }  
}
```

(4) (4 分)

```
int main( ) {  
    int n, i;  
    struct Time Pt, Gt, Cs[100];  
    printf("Please enter the number of crops and the planting time: ");  
    scanf("%d %d:%d", &n, &Pt.H, &Pt.M);  
    for (i=0; i<n; i++) {  
        scanf("%d:%d", &Gt.H, &Gt.M);  
        Cal(&Pt, &Gt, &Cs[i]);  
    }  
    Sort(Cs, n);  
    for (i=0; i<n; i++) {  
        printf("%d:%d\n", Cs[i].H, Cs[i].M);  
    }  
    return 0;  
}
```

3. (14 分)

struct LA { //LA 链表结点定义正确, 1 分

int d;

struct LA *next;

};

struct LB { //LB 链表结点定义正确, 1 分

int d1, d2;

struct LB *next;

};

int gcd(int a, int b) { //最大公约数求法正确 (可以不写成函数), 2 分

int t;

while (a != 0) {

t = a; a = b % a; b = t;

}

return b;

}

struct LB* Pair(struct LA* A) { //返回值类型, 1 分, 参数类型, 1 分

struct LB* B = NULL, *temp1, *temp; //B 初始化为 NULL, 1 分

struct LA* p = A, *q = A;

while (p != NULL) { //外重循环 (必须有 p=p->next), 1 分

q = p->next;

while (q != NULL) { //内重循环 (必须有 q=q->next), 1 分

if (gcd(p->d, q->d)==1) {

temp = (struct LB*)malloc(sizeof(struct LB)); //malloc 1 分,

sizeof 1 分, 强制转换不计分

temp->d1 = p->d;

temp->d2 = q->d;

temp->next = NULL;

if (B == NULL) //插入结点逻辑, 2 分 (尾插或者头插均可)

B = temp;

else

temp1->next = temp;

temp1 = temp;

}

q = q->next;

}

p = p->next;

}

return B; //返回 B, 1 分

}